

Debugging in the Browser – Simple Hinglish Summary

Source: <https://javascript.info/debugging-chrome>

Main baat

Debugging matlab script ke **errors dhundhna aur theek karna**. Saare modern browsers me debugging tools hote hain (developer tools ka special UI). Isse code ko **step-by-step trace** karke dekh sakte hain ki exactly kya ho raha hai.

Yahan Chrome use hua hai, baaki browsers me process lagbhag same hai.

1. Sources panel

1. Example page Chrome me kholo.
2. Developer tools ON karo: **(F12)** (Mac: **(Cmd + Opt + I)**).
3. **Sources** panel select karo.

Toggler button se files ka tab khulta hai. Wahan se **(hello.js)** chuno.

Sources panel ke 3 hisse:

1. **File Navigator**: HTML, JS, CSS aur images jaisi saari files ki list.
2. **Code Editor**: source code dikhata hai.
3. **JavaScript Debugging pane**: debugging ke liye (aage detail me).

2. Console

(Esc) dabane par neeche **console** khulta hai. Wahan commands type karke **(Enter)** dabao to wo chalti hain, aur result neeche dikhta hai.

Jaise **(1+2)** ka result **(3)**. Function call jo kuch return nahi karta, uska result **(undefined)** hota hai.

3. Breakpoints

Breakpoint code ka wo point hai jahan debugger **JavaScript execution ko automatically rok deta hai**.

Kaise lagayein: **(hello.js)** me line number par click karo (code par nahi, number par).

Code ruka hone par:

- Current variables dekh sakte ho
- Console me commands chala sakte ho
- Yaani debug kar sakte ho

Right panel me saare breakpoints ki **list** hoti hai. Isse:

- Breakpoint par turant jump kar sakte ho

- Uncheck karke temporarily disable kar sakte ho
- Right-click karke Remove kar sakte ho

Conditional breakpoints

Line number par **right click** karke **conditional breakpoint** bana sakte ho. Ye tabhi rukta hai jab di gayi expression **truthy** ho. Jab sirf kisi khaas variable value par rukna ho, tab kaam aata hai.

4. `debugger` command

Code me `debugger;` likhkar bhi pause kar sakte ho:

```
javascript

function hello(name) {
  let phrase = `Hello, ${name}!`;

  debugger; // <-- yahan debugger ruk jayega

  say(phrase);
}
```

Dhyan: Ye tabhi kaam karta hai jab **developer tools khule hon**, warna browser ise ignore kar deta hai.

5. Pause karke aas-paas dekhna

Breakpoints lagane ke baad page **reload** karo (`F5`, Mac: `Cmd + R`). Execution breakpoint par ruk jayega.

Right side ke dropdowns se code ki current state dekh sakte ho:

1. **Watch:** kisi bhi expression ki current value dikhata hai. `+` dabakar expression daalo. Execution ke saath value automatically dobara calculate hoti hai.
2. **Call Stack:** nested calls ki chain dikhata hai. Kisi item par click karo to debugger us code par jump karta hai aur uske variables bhi dekh sakte ho.
3. **Scope:** current variables.
 - **Local:** function ke local variables
 - **Global:** function ke bahar ke variables
 - `this` bhi dikhta hai (abhi nahi seekha)

6. Execution trace karna

Right panel ke top par buttons hote hain:

Button	Hotkey	Kya karta hai
Resume	F8	Execution jaari rakhta hai. Aur breakpoint na ho to debugger control chhod deta hai
Step	F9	Agla statement chalata hai. Baar baar dabane par saare statements ek ek karke chalte hain
Step over	F10	Agla command chalata hai, lekin function ke andar nahi jata . Function ko "chupke se" chalakar uske baad ruk jata hai
Step into	F11	Step jaisa, par async calls (jaise <code>setTimeout</code>) me alag behave karta hai. Abhi ignore kar sakte ho
Step out	Shift + F11	Current function ke aakhri line tak chalata hai. Galti se nested call me ghus gaye ho to kaam aata hai
Enable/disable all breakpoints		Sirf breakpoints ko mass on/off karta hai, execution nahi hilata
Pause on error		ON ho to error aane par script automatically ruk jati hai (dev tools khule ho to). Isse pata chalta hai script kahan aur kis context me fail hui

Step vs Step over: Agla statement apne khud ke function ka call ho, to **Step** us function ke andar jata hai (pehli line par ruk kar), jabki **Step over** function ko skip karke uske baad wali line par rukta hai.

"Continue to here"

Code ki line par **right click** karo, "Continue to here" option milta hai. Kai steps aage jana ho aur breakpoint lagane ka mann na ho, tab handy hai.

7. Logging (`console.log`)

Code se console me kuch print karne ke liye `console.log` use hota hai:

```
javascript
for (let i = 0; i < 5; i++) {
  console.log("value,", i);
}
```

- Normal users ko ye output nahi dikhta, ye sirf **console** me hota hai.
- Dekhne ke liye Console panel kholo ya kisi aur panel me **Esc** dabao.

- Agar code me kaafi logging ho, to debugger ke bina bhi records se samajh sakte ho ki kya chal raha hai.

Summary

Script rokne ke **teen main tarike**:

1. **Breakpoint**
2. `debugger` **statement**
3. **Error** (agar dev tools khule ho aur pause-on-error button ON ho)

Rukne ke baad variables examine kar sakte ho aur code trace karke dekh sakte ho ki execution kahan galat ja raha hai.

Dev tools me aur bhi bahut options hain. Full manual:

<https://developers.google.com/web/tools/chrome-devtools>

Tip: Dev tools seekhne ka sabse tez tarika hai alag alag jagah click karke dekhna. Right click aur context menus ko na bhoolein.

Quick Summary

Kaam	Kaise
Dev tools kholna	<code>F12</code> (Mac: <code>Cmd + Opt + I</code>)
Console kholna	<code>Esc</code>
Breakpoint lagana	Line number par click
Code me pause	<code>debugger;</code>
Aage badhna	<code>F8</code> Resume, <code>F9</code> Step, <code>F10</code> Step over
Function se bahar	<code>Shift + F11</code> Step out
Print karna	<code>console.log(...)</code>